Design And Analysis of Algorithms
Lecture 9: Algebraic Methods for Matchings

February 27, 2018

In this lecture, we consider yet another way to find matchings in graphs: this time, the focus will be on *algebraic* methods for finding perfect matchings. All of these algorithms are based on the same broad idea: construct some matrix M_G such that evaluating the determinant of M_G gives a test for whether or not the graph contains a perfect matching. The trick will be to figure out what M_G should be. **For the remainder of this lecture, assume $\tilde{f}$ means f applied to arguments chosen uniformly at random.**

1 Roots of low-degree polynomials

It will be useful to have the following facts when we start talking about algebraic algorithms. The first reminds us that any degree-d polynomial besides the zero polynomial can have at most d roots.

Theorem 1. *Any polynomial p in one dimension of degree at most d has at most d roots, if $p \neq 0$.*

Multinomials don't have quite the same generosity of a guarantee: consider $p(x, y) = xy$, a degree two multinomial with uncountably many zeros (whenever either $x = 0$ or $y = 0$). These are relatively sparse when compared to the entirety of $\mathbb{R}^2$; the following theorem captures that precise relationship.

Theorem 2. *[[Schwartz, 1980, DeMillo and Lipton, 1977, Zippel, 1979]] Consider a multinomial p of degree at most d over k variables in some field $\mathbb{F}$. Let $S \subseteq \mathbb{F}$. Then, select $X_1, \ldots, X_k$ uniformly at random and independently from S. If p is not the zero polynomial,*

$$\mathbb{P}_{X_1,\ldots,X_n \sim_{UAR} S}\left[p(X_1, \ldots, X_n) = 0\right] \leq \frac{d}{|S|}$$

Proof. The proof will be by induction on n, the number of independent variables for p. The base case, when $n = 1$, is implied by Theorem 1.

Now, consider some $n > 1$, and assume for all $n' < n$ the theorem holds. Suppose that the dimension of p is d. Let q, r be defined such that

$$p = x_n^k \cdot q + r$$

where r is guaranteed to have degree at most $k - 1$ in x_n and q does not depend on x_n. Note that

$$\begin{aligned}
\mathbb{P}[\tilde{p} = 0] &= \mathbb{P}[\tilde{p} = 0 | \tilde{q} = 0]\mathbb{P}[\tilde{q} = 0] + \mathbb{P}[\tilde{p} = 0 | \tilde{q} \neq 0]\mathbb{P}[\tilde{q} \neq 0] \\
&\leq \mathbb{P}[\tilde{q} = 0] + \mathbb{P}[\tilde{p} = 0 | \tilde{q} \neq 0] \\
&\leq \frac{d-k}{|S} + \frac{k}{|S|}
\end{aligned}$$

where the last line follows from the inductive hypothesis applied to q, which is a degree (at most) $d - k$ polynomial, and the fact that p can be viewed as a monomial of degree at most k fixing the first $n - 1$ input variables. □

2 Perfect Matchings in a Bipartite Graph, an $O(m \cdot n^\omega)$ algorithm of Lovász [1979]

For our first example of this technique, consider a bipartite graph $G = (L, R, E)$ for which we want to know if a perfect matching exists. Suppose for ease of notation $L = \{\ell_1, \ldots, \ell_n\}$ and $R = \{r_1, \ldots, r_n\}$. Define variables x_{ij} for each $i, j \in [n]$, which will belong to the multinomials we consider shortly. We define the matrix known as Edmonds' matrix, $\epsilon(G)$, such that

$$\epsilon(G)_{ij} = \begin{cases} x_{i,j} & (\ell_i, r_j) \in E \\ 0 & \text{otherwise.} \end{cases}$$

We will now use an invaluable calculation over matrices to evaluate the Edmonds matrix.

Definition 1. *Given an $n \times n$-dimensional real-valued matrix A, the* determinant *of A, written* $\det(A)$*, is defined to be*

$$\sum_{\sigma \in S_n} (-1)^{sign(\sigma)} \prod_{i \in [n]} A_{i\sigma(i)}.$$

Notice that we can consider $\det(\epsilon(G))$ as a polynomial over the n^2 variables used to define $\epsilon(G)$ using exactly this definitional form.

Observation 1. *G has a perfect matching if and only if* $\det(\epsilon(G))$ *is a nonzero polynomial.*

Proof. Since each possible matching can be thought of as a permutation on n, each element of the sum refers to some possible matching. If that term corresponds to a matching which exists, none of $\epsilon(G)_{i,\sigma(i)}$ will be zero. Finally, since each element of $\epsilon(G)$ is either 0 or a unique variable, no terms in the sum will cancel one another. □

Algorithm 1: Test-PM(G, S)

Choose $X_{i,j}$ uniformly at random from S;
Evaluate $d = \det(\epsilon(G)(X_{11}, \ldots, X_{nn}))$;
Return "true" if $d \neq 0$;
Return "false" if $d = 0$.

Notice that the runtime of Find-PM is $O(mn^\omega)$, where n^ω is the runtime of matrix multiplication which suffices to solve for determinants [Bunch and Hopcroft, 1974]. It can be improved to $O(n^{omega+1} \ln(n))$ by instead iterating over vertices and doing a "binary search" for some edge belonging to a perfect matching.

Theorem 3. *With probability at least $\frac{1}{2}$, if $|S| > n^3$, Find-PM returns a perfect matching if one exists. If one does not exist, it always returns false.*

Proof. This follows from a union bound and Theorem 2 along with observation 1 and the fact that $m \leq n^2/2$. □

Remark 1. *One can convert the above into a high probability bound merely by repeating it $c \ln n$ times.*

3 Non-Bipartite Graphs and Perfect Matchings: the Tutte Matrix [Lovász, 1979]

For non-bipartite graphs, one needs a different matrix to follow the same blueprint of evaluating the determinant at random points drawn over $\mathbb{R}$. We define $T(G)$, the Tutte matrix, as follows:

Algorithm 2: Find-PM(G, S)

```
if Test-PM(G, S) then
    for each e = (u, v) ∈ E do
        if Test-PM(G \ e, S) then
            Find-PM(G \ e, S)
        end
        else
            return e ∪ Find-PM(G \ {u, v}, S);
        end
    end
end
else
    return "false";
end
```

$$T(G)_{ij} = \begin{cases} x_{i,j} & (i < j) \wedge (i,j) \in E \\ -x_{i,j} & (i > j) \wedge (i,j) \in E \\ 0 & \text{otherwise.} \end{cases}$$

This matrix then captures something about the connectivity of G even without the bipartite structure of the previous section. The following is left as an exercise for the reader.

Theorem 4. $\det(T(G))$ *is the zero polynomial if and only if G does not contain a perfect matching. Thus, one can find a perfect matching in time $\tilde{O}(m \cdot n^{\omega})$ if one exists, with high probability.*

4 Rabin-Vazirani $n^{\omega+1}$-algorithm of Rabin and Vazirani [1989]

We now present an algorithm due to Rabin and Vazirani [1989] for finding a perfect matching. This algorithm works for both bipartite and non-bipartite graphs, but we focus on the bipartite case for the remainder of the lecture. The algorithm behaves much like the above algorithms: it considers the Edmonds/Tutte matrix, and looks at its determinant. The improvement in speed comes from determining whether or not the determinant is zero. Intuitively, if some perfect matching exists, say π permutation π, then $\epsilon(G)_{i,\pi(i)} \neq 0$ for all $i \in [n]$.

This naive implementation does $O(n)$ many determinant computations in each iterative call, of which there might be n. So, this algorithm has runtime $O(n^2 \cdot n^{\omega})$, worse than the other algorithms we've considered today. However, it can be easily sped up by using the (arguably more useful) definition of determinants:

Theorem 5 (Cramer's Rule)**.** *For any $n \times n$ real-valued matrix A, we have that*

$$\det(A^{-}1)_{ij} = \frac{(-1)^{i+j} \det(A_{-i,ij})}{\det(A)}.$$

Thus, one can do only one expensive determinant computation per recursive call, which gives us the following theorem

Theorem 6. *The above algorithm can be implemented to find a perfect matching in a bipartite graph G with probability at least $\frac{1}{2}$ if one exists in time $O(n^{1+\omega})$.*

Algorithm 3: Naive-PM(G, S)

```
Construct ϵ(G);
Sample R_ij ~ S;
Let ε̃ = ϵ(G)(R);
Compute ε̃^{-1}, det ε̃ if det(ε̃) = 0 then
  | Return "no matching"
end
else
  | for j = 1 to n do
  |   | if ε̃_{1,j} ≠ 0 and det(ε̃_{-1,-j}) ≠ 0 then
  |   |   | Return (ℓ_1, r_j) ∪ Naive-PM(G[V − {ℓ_1, r_j}], S)
  |   | end
  | end
end
```

References

James R Bunch and John E Hopcroft. Triangular factorization and inversion by fast matrix multiplication. *Mathematics of Computation*, 28(125):231–236, 1974.

Richard A DeMillo and Richard J Lipton. A probabilistic remark on algebraic program testing. Technical report, GEORGIA INST OF TECH ATLANTA SCHOOL OF INFORMATION AND COMPUTER SCIENCE, 1977.

László Lovász. On determinants, matchings, and random algorithms. In *FCT*, volume 79, pages 565–574, 1979.

Silvio Micali and Vijay V Vazirani. An o (v— v— c— e—) algoithm for finding maximum matching in general graphs. In *Foundations of Computer Science, 1980., 21st Annual Symposium on*, pages 17–27. IEEE, 1980.

Marcin Mucha and Piotr Sankowski. Maximum matchings via gaussian elimination. In *Foundations of Computer Science, 2004. Proceedings. 45th Annual IEEE Symposium on*, pages 248–255. IEEE, 2004.

Ketan Mulmuley, Umesh V Vazirani, and Vijay V Vazirani. Matching is as easy as matrix inversion. In *Proceedings of the nineteenth annual ACM symposium on Theory of computing*, pages 345–354. ACM, 1987.

Michael O Rabin and Vijay V Vazirani. Maximum matchings in general graphs through randomization. *Journal of Algorithms*, 10(4):557–567, 1989.

Jacob T Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)*, 27(4):701–717, 1980.

Richard Zippel. Probabilistic algorithms for sparse polynomials. In *Symbolic and algebraic computation*, pages 216–226. Springer, 1979.